SESSION 3

Feature Engineering for Text in NLP

Definitions • Examples • Dataset • Practical Workflow • Exercises • Interview Questions

Learning objective
Understand how raw text is converted into numerical features using Bag of Words (BoW) and TF-IDF, and learn how to explore a real customer-review dataset.

Code note: This PDF includes small illustrative snippets and the practical workflow, but intentionally excludes the full Python implementation for BoW and TF-IDF, as requested.

NLP Training Notes

1. What Is Feature Engineering in NLP?

Definition: Feature engineering in Natural Language Processing (NLP) is the process of converting raw text into meaningful numerical representations that machine-learning algorithms can understand and process.

Traditional machine-learning algorithms generally require numerical input. Therefore, text such as customer reviews must be represented as numbers before it can be used as model features.

Example

Review ID	Customer review	Sentiment
1	This product is good	Positive
2	This product is bad	Negative
3	Product quality is good	Positive
4	This is a bad product	Negative
5	Good quality product	Positive

A text vector might look like `[1, 1, 1, 1, 0, 0]`. Each position corresponds to a word in a vocabulary; a value indicates whether, or how often, that word appears.

Why is feature engineering useful?

• Sentiment analysis and customer-review classification

• Spam email detection

• News categorization and document classification

• Customer feedback analysis

• Document similarity and information retrieval

2. Converting Text into Numbers

Text vectorization represents each document or sentence as a numerical vector. A vector has one position for each feature in the vocabulary.

Example sentences:

```
sentences = [\n    "I like Python",\n    "I like NLP",\n    "Python is good"\n]
```

A manually created vocabulary could be: **I, like, Python, NLP, is, good**.

Sentence	I	like	Python	NLP	is	good
I like Python	1	1	1	0	0	0
I like NLP	1	1	0	1	0	0
Python is good	0	0	1	0	1	1

The table is an example of a document-term matrix: rows are documents, columns are vocabulary terms, and cells contain numerical values.

3. What Is Bag of Words (BoW)?

Definition: Bag of Words is a text-representation technique that converts text into numerical vectors by counting the occurrences of words in each document. Standard BoW does not preserve word order or grammar.

Example

Sentences: `I love Python`, `I love NLP`, and `Python is good`.

Sentence	I	love	Python	NLP	is	good
I love Python	1	1	1	0	0	0
I love NLP	1	1	0	1	0	0
Python is good	0	0	1	0	1	1

Repeated words: In the sentence “Python is good and Python is popular,” the word Python occurs twice, so its BoW count is 2.

Limitation: “dog bites man” and “man bites dog” have the same word counts, although their meanings differ.

4. What Is Vocabulary in NLP?

Definition: A vocabulary is the collection of unique terms identified from the text corpus during vectorization.

For the sentences “I love Python,” “I love NLP,” and “Python is good,” a manually created vocabulary is: I, love, Python, NLP, is, good. Its size is 6.

Example using scikit-learn:

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "I love Python",
    "I love NLP",
    "Python is good"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())
```

By default, CountVectorizer ignores one-character tokens and removes common English stop words only if stop-word removal is configured. For this example, the default tokenization may omit “I” and the default English stop-word list is not applied unless requested.

> To allow one-character tokens, use `token_pattern=r'(?u)\b\w+\b'`. This does not itself disable any separately configured stop-word removal.

5. What Is Word Frequency?

Definition: Word frequency is the number of times a particular word appears in a document or text dataset.

Example sentence: “Python is easy. Python is powerful. Python is popular.”

Word	Frequency
Python	3
is	3
easy	1
powerful	1
popular	1

Word frequency helps identify commonly discussed topics, explore customer feedback, and understand the terms that appear most often.

Limitation: A frequent word is not necessarily informative. Words such as “the” and “is” can appear often but may not distinguish one document from another.

6. What Is TF-IDF?

Definition: TF-IDF stands for Term Frequency–Inverse Document Frequency. It measures how important a word is to a document relative to a collection of documents.

TF-IDF tends to give higher weight to terms that occur in a document but are less common across the full collection.

A. Term Frequency (TF)

A basic term-frequency formula is:

TF(t, d) = count of term t in document d ÷ total number of words in document d

Example document: “Python is good Python is easy.” There are 6 words and Python appears twice.

TF(Python) = 2 ÷ 6 = 0.3333

B. Inverse Document Frequency (IDF)

IDF measures how uncommon a term is across the collection. A common smoothed formula is:

IDF(t) = log((N + 1) ÷ (DF(t) + 1)) + 1

• **N:** total number of documents

• **DF(t):** number of documents containing term t

• **log:** natural logarithm

Example: there are 5 documents, and Python appears in 2 of them.

IDF(Python) = log((5 + 1) ÷ (2 + 1)) + 1 = log(2) + 1 ≈ 1.6931

C. Calculate TF-IDF

TF-IDF = TF × IDF

Using TF = 0.3333 and IDF = 1.6931: TF-IDF ≈ 0.3333 × 1.6931 ≈ 0.5644.

This is an illustrative calculation using the formulas shown. Scikit-learn’s TfidfVectorizer uses L2 normalization by default, so the final vector values can differ.

7. Difference Between Bag of Words and TF-IDF

Feature	Bag of Words (BoW)	TF-IDF
Purpose	Represents text using word counts	Represents text using weighted word values
Frequency	Counts word occurrences	Considers term frequency and document rarity
Common words	Can receive high counts	Usually receive lower IDF weights when widespread
Output	Integer counts	Usually decimal values
Complexity	Simple	Slightly more complex
Applications	Basic text classification and word counts	Search, document ranking and text classification

Example documents: “Python is useful,” “Python is powerful,” and “Python is popular.”

• BoW counts Python once in each document.

• TF-IDF gives Python a relatively low IDF weight because it occurs in every document.

• Words such as useful, powerful, and popular occur in only one document each and may receive higher IDF weights.

TF-IDF does not automatically understand sentiment or meaning; it measures importance using frequency and document distribution.

8. Practical Example: Customer Review Text Analysis

Project and business problem

An e-commerce company receives customer reviews and wants to convert them into numerical features for text analysis and, later, sentiment classification.

Dataset

Use the **Women's E-Commerce Clothing Reviews** dataset from Kaggle. It contains customer review text and review information, including a "Review Text" field and a "Rating" field.

Dataset page: https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

Download instructions

• Open the Kaggle dataset page.

• Download the dataset and extract the ZIP file.

• Locate the CSV file, commonly named "Womens Clothing E-Commerce Reviews.csv."

• Place the CSV file in your Python project's working directory.

Install the required libraries:

```
pip install pandas scikit-learn matplotlib
```

9. Practical Workflow: BoW and TF-IDF

The complete BoW and TF-IDF Python implementation is intentionally omitted from this PDF, as requested. Use the following workflow as a practical guide.

Step	Action	Purpose
1	Load the CSV using pandas	Read the review dataset
2	Select Review Text and Rating	Keep relevant columns
3	Inspect missing values	Identify incomplete reviews
4	Remove rows with missing review text	Prepare valid text records
5	Convert reviews to lowercase	Normalize letter case
6	Optionally remove duplicate reviews	Avoid repeated text in the demo
7	Select a small sample, such as 100 reviews	Keep the demonstration manageable
8	Create CountVectorizer	Generate BoW word-count features
9	Inspect vocabulary and matrix shape	Understand features and dimensions
10	Sum each vocabulary column	Calculate word frequencies
11	Plot the top 10 words	Visualize common terms
12	Create TfidfVectorizer	Generate TF-IDF features
13	Inspect a review's nonzero values	Interpret the numerical vector
14	Compare matrix shapes and values	Contrast counts with weighted features
15	Save the feature matrices as CSV	Reuse features in later exercises

Important implementation notes

• `CountVectorizer()` learns a vocabulary and creates word-count vectors.

• `TfidfVectorizer()` learns a vocabulary and creates TF-IDF vectors.

• `fit_transform()` learns the vocabulary and transforms documents.

• `get_feature_names_out()` returns the learned feature names.

• `max_features=100` limits the vocabulary to at most 100 features.

• `stop_words='english'` removes common English stop words.

• Converting a large sparse matrix to a dense array can use substantial memory. Avoid doing so for very large datasets.

• Exact vocabulary, frequencies, and TF-IDF values depend on the dataset and selected sample.

10. Understanding the Output

Suppose a review says: “This dress is beautiful and comfortable.” After common English stop words are removed, terms might include “dress,” “beautiful,” and “comfortable,” depending on the vectorizer settings.

Illustrative BoW output

Word	Count
beautiful	1
comfortable	1
dress	1
quality	0
size	0

Illustrative vector: `[1, 1, 1, 0, 0]`.

Illustrative TF-IDF output

Word	Illustrative TF-IDF
beautiful	0.58
comfortable	0.58
dress	0.58
quality	0.00
size	0.00

These decimal values are examples only, not results from the Kaggle dataset. Actual values depend on the vocabulary, document frequencies, and vector normalization.

BoW records counts, while TF-IDF weights terms based on their frequency in a document and their distribution across the corpus.

11. Practical Exercise for Students

Problem statement: An online clothing retailer wants to analyze customer reviews and prepare text data for a sentiment-classification model. Use the provided customer-review dataset to perform feature engineering.

Tasks

• Load the CSV dataset using Pandas and display the first five records.

• Check the dataset shape.

• Identify missing values in the Review Text column.

• Remove records with missing review text.

• Convert review text to lowercase.

• Create a Bag of Words vectorizer and extract the vocabulary.

• Display vocabulary size.

• Calculate word frequencies and display the top 10 most frequent words.

• Plot the top 10 words using a bar chart.

• Create a TF-IDF vectorizer and generate the TF-IDF matrix.

• Display TF-IDF values for one review.

• Compare BoW and TF-IDF representations.

• Save both feature matrices as CSV files.

Expected learning outcomes

• Explain why raw text must be converted into numerical features.

• Create and inspect a vocabulary.

• Calculate word frequency.

• Generate word-count vectors using BoW.

• Explain how TF-IDF weights words using document distribution.

• Prepare text features for a later machine-learning task.

12. Interview Questions and Answers

Q1. What is feature engineering in NLP?

It converts raw text into numerical features that machine-learning algorithms can process.

Q2. What is Bag of Words?

A technique that represents text as vectors based on word counts.

Q3. What is vocabulary in NLP?

The collection of unique terms learned from a text corpus.

Q4. What is word frequency?

The number of times a word appears in a document or dataset.

Q5. What is TF-IDF?

A weighting technique based on a term’s frequency in a document and how widely it occurs across documents.

Q6. What is the difference between BoW and TF-IDF?

BoW uses word counts; TF-IDF uses weighted values based on term frequency and inverse document frequency.

Q7. Does BoW preserve word order?

No. Standard BoW does not preserve word order or grammar.

Q8. Why is TF-IDF useful?

It can highlight terms that are relatively informative for particular documents.

Q9. What does CountVectorizer() do?

It learns a vocabulary and converts documents into word-count vectors.

Q10. What does TfidfVectorizer() do?

It learns a vocabulary and converts documents into TF-IDF vectors.

13. Session Summary

Concept	Meaning
Feature engineering	Converting raw text into useful numerical features
Text vectorization	Representing text as numerical vectors
Vocabulary	Collection of unique words or terms
Word frequency	Number of occurrences of a word
Bag of Words	Numerical representation based on word counts
TF	Measures a term's frequency in a document
IDF	Measures how uncommon a term is across documents
TF-IDF	Combines term frequency and inverse document frequency
CountVectorizer()	Creates word-count features
TfidfVectorizer()	Creates TF-IDF features

Recommended teaching sequence: First demonstrate vocabulary creation and word counts with three simple sentences. Then use the clothing-review dataset to generate BoW and TF-IDF vectors. Finally, explain how these features can be used in a machine-learning model for sentiment classification.